

Plan de Desarrollo: Sistema de Gestión y Registro de Reclamos

Como tu **mentor en este proyecto**, abordaremos el desarrollo con las mejores prácticas de la industria, estructurando el trabajo por etapas claras y comprensibles. Iremos avanzando paso a paso para que domines cada herramienta: desde la contenedorización con **Docker** y el diseño relacional en **PostgreSQL**, hasta la lógica de backend, la interfaz de usuario moderna y el despliegue para producción.

1. Visión General de la Arquitectura

```
flowchart TD
    subgraph Cliente ["Frontend (Navegador)"]
        UI["SPA / Interfaz Web Moderna\nHTML5 + CSS + Van"]
        P1["1. Gestión Reclamos"]
        P2["2. Historial + Filtros + PDF"]
        P3["3. Registros de Auditoría"]
        P4["4. Edición de Estructura"]
        P5["5. Perfil & Gestión Usuarios"]
        UI --> P1 & P2 & P3 & P4 & P5
    end

    subgraph DockerHost ["Entorno Docker Compose"]
        subgraph BackendContainer ["Contenedor App (Node.js)"]
            Auth["Módulo Auth (JWT & Roles)"]
            API["REST API (Reclamos, Catálogos, Auditoría)"]
            PDFGen["Motor de Reportes PDF"]
            AuditMiddleware["Interceptor de Auditoría"]
        end
    end
```

```

        subgraph DBContainer ["Contenedor Base de Datos"]
            PG[(PostgreSQL 16)]
            Volume[(Volumen Persistente 'pgdata')]
            PG --- Volume
        end
    end

    Cliente <-->|HTTP / REST API / JSON| BackendContainer
    BackendContainer <-->|pg pool / SQL| DBContainer

```

2. Modelo de Datos Relacional (PostgreSQL)

Analizando los datos y la planilla adjunta, la estructura de reclamos es **jerárquica y en cascada**:

1. **Tipo de Consulta** (ej: *Reclamos Técnicos, Facturación, Venta, Baja, Consultas Generales, Baja Parcial*)
2. **Características de la Consulta** (vinculada a Tipo de Consulta, ej: *Inconveniente catv, Inconveniente conexión, Pago, Mudanza*)
3. **Definición** (vinculada a Característica, ej: *Sin señal, Lentitud / Cortes, Informe de pago, Consulta cambio de plan*)
4. **Finalización** (vinculada al flujo correspondiente o catálogo maestro, ej: *Resuelto, OT, Pendiente, Informado, Retenido, No retenido*)

Entidades Principales:

- **usuarios:** id, nombre_completo, documento (UNIQUE), usuario (UNIQUE), password_hash, foto_perfil, legajo, rol ('Admin', 'Supervisor', 'Asesor', 'Espectador'), activo, created_at.
- **sucursales:** id, nombre, activo, created_at.
- **tipos_consulta:** id, nombre, descripcion, activo.
- **caracteristicas_consulta:** id, tipo_consulta_id (FK),

- nombre, descripcion, activo.
- definiciones_consulta: id, caracteristica_id (FK), nombre, descripcion, activo.
- finalizaciones: id, definicion_id (FK opcional para cascada estricta o relación N:M / catálogo según necesidad), nombre, descripcion, activo.
- reclamos: id, fecha (TIMESTAMP), sucursal_id (FK), usuario_id (FK asesor), numero_cliente (VARCHAR(15)), tipo_consulta_id (FK), caracteristica_id (FK), definicion_id (FK nullable), finalizacion_id (FK nullable), created_at, updated_at.
- auditoria_logs: id, fecha, accion (VARCHAR), usuario_id (FK), usuario_nombre (VARCHAR), entidad (VARCHAR), registro_id (INTEGER), datos_anteriores (JSONB / TEXT), created_at.

3. Matriz de Roles y Permisos

Módulo / Pestaña	Admin	Supervisor	Asesor	Espectador
Gestión de Reclamos (Carga)	Total	Total	Total	Sin acceso
Historial (Lectura y Filtros)	Total	Total	Total	Solo lectura
Historial (Editar Reclamo)	Sí	Sí	Sí	No
Historial (Eliminar Reclamo)	Sí	No	No	No

Historial (Emitir Reporte PDF)	Sí	Sí	Sí	No
Registros (Auditoría con Paginación)	Sí	No	No	No
Edición (Estructura y Sucursales)	Total	Total	No	No
Perfil (Cambio clave / foto)	Propio	Propio	Propio	Propio
Gestión de Usuarios en Perfil	Total (Crear, Editar, Borrar cualquier rol)	Crear solo Asesores, sin borrar superiores	No	No

4. Hoja de Ruta Paso a Paso (Roadmap de Mentoría)

Fase 1: Infraestructura y Base de Datos con Docker

- Configuración de Docker Compose (`docker-compose.yml`):**
 - Servicio PostgreSQL con imagen oficial `postgres:16-alpine`.
 - Volumen de datos persistente para que los datos nunca se pierdan al apagar el contenedor.
 - Variables de entorno (`.env`) para credenciales seguras.
- Scripts de inicialización SQL (`init.sql` / migraciones):**

- Creación de tablas e índices para optimizar búsquedas.
- Inserción de datos semilla iniciales (usuario Admin por defecto, sucursales y todo el árbol de tipos/características/definiciones/finalizaciones extraído de la planilla adjunta).

3. Validación del contenedor y conexión SQL.

Fase 2: Backend y API REST (Node.js + Express + PostgreSQL)

1. Configuración del proyecto backend estructurado en capas (Rutas, Controladores, Modelos/Servicios, Middlewares).
2. Conexión a PostgreSQL mediante `pg` (Connection Pooling).
3. Autenticación con JWT (JSON Web Tokens) y contraseñas seguras con `bcrypt`.
4. Middleware de control de accesos basado en roles (RBAC).
5. Middleware automático de Auditoría: cada vez que se modifique o elimine un reclamo o usuario, se registra el estado previo en `auditoria_logs`.
6. Endpoints de la API:
 - `/api/auth`: Login, validación de sesión.
 - `/api/reclamos`: CRUD, filtros avanzados y paginación.
 - `/api/catalogos`: Gestión de sucursales y árbol jerárquico de consultas.
 - `/api/usuarios`: Gestión de perfiles, roles y fotos de perfil (con `multer`).
 - `/api/auditoria`: Consulta de logs con paginación de 20 registros.
 - `/api/reportes/pdf`: Generación de reporte descargable en PDF con los filtros aplicados.

Fase 3: Frontend (Interfaz Web Moderna y Responsive)

1. Arquitectura Visual:
 - Barra superior con datos del usuario logueado, foto de perfil, rol y botón de cierre de sesión.
 - Navegación por pestañas dinámicas (visibles únicamente según el rol del usuario).

2. Pestaña 1 - Gestión de Reclamos:

- Formulario ágil con selects en cascada dependientes (Tipo -> Característica -> Definición -> Finalización).
- Carga automática de fecha actual y asesor logueado.

3. Pestaña 2 - Historial:

- Panel de filtros combinables (Fecha desde/hasta, Asesor, Sucursal, Tipo de consulta).
- Tabla interactiva con acciones de editar y eliminar (según permisos).
- Botón para exportar el listado filtrado a PDF.

4. Pestaña 3 - Registros de Auditoría:

- Tabla paginada de a 20 registros con detalles de cambios anteriores en modal o acordeón.

5. Pestaña 4 - Edición:

- Interfaz de mantenimiento de Sucursales y de la jerarquía de Reclamos (crear, editar, activar/desactivar).

6. Pestaña 5 - Perfil & Usuarios:

- Ver perfil, cambio de contraseña con validación de clave actual, subida de foto de perfil.
- Sección de administración de usuarios (para Admin y Supervisor) con tabla y modales de creación/edición.

Fase 4: Pruebas, Verificación y Auditoría

1. Pruebas funcionales de los 4 roles.
2. Pruebas de cascada de datos y restricciones de integridad.
3. Validación de generación de reportes PDF y logs de auditoría.

Fase 5: Despliegue y Puesta en Línea

1. Dockerización completa del frontend y backend en contenedores listos para producción.
 2. Opciones de montaje en red local (LAN) o en la nube (VPS con Docker, Nginx reverse proxy y SSL/HTTPS).
-

5. Plan de Verificación

Pruebas de Base de Datos y Docker:

- Levantar `docker compose up -d` y verificar estado con `docker ps` y logs de base de datos.
- Ejecutar queries de prueba para confirmar relaciones y claves foráneas.

Pruebas de API y Permisos:

- Iniciar sesión con Admin, Supervisor, Asesor y Espectador y verificar qué endpoints responden y cuáles bloquean con 403 Forbidden.
- Modificar un reclamo y verificar que la tabla `auditoria_logs` guarde el estado anterior exacto.

Pruebas de Interfaz de Usuario:

- Navegación entre las 5 pestañas según rol.
- Verificación del comportamiento en cascada de los selects de reclamo.
- Paginación de 20 registros en la pestaña de Registros.
- Emisión de PDF con filtros seleccionados.